



Rajarata University of Sri Lanka

Department of Computing

ICT1305 – Data Structures
Laboratory 04 – Linked list in C
Time: 120 Minutes

Objective:

By the end of this lab, students should be able to:

- Implement advanced functions in singly linked lists.
- Sort elements in ascending order.
- Merge two linked lists.
- Shortlist (filter) nodes based on a given condition.
- Count the number of occurrences of a given value.
- Reverse a linked list.
- Remove duplicate values.
- Reinforce best practices in memory handling and pointer operations.

Function 1: Create and Display a Linked List

This program demonstrates how to create a basic singly linked list with three nodes and display its elements using a loop. Each node is dynamically allocated, and the `displayList` function is used to traverse and print the list.

Question: Write a program to create a singly linked list with three nodes, and display its content.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}
```

```

void displayList(struct Node* head) {
    while (head != NULL) {
        printf("%d -> ", head->data);
        head = head->next;
    }
    printf("NULL\n");
}

int main() {
    struct Node* head = createNode(10);
    head->next = createNode(20);
    head->next->next = createNode(30);
    displayList(head);
    return 0;
}

```

Function 2: Sort Linked List in Ascending Order

This function sorts the linked list in ascending order by comparing and swapping the data values using a basic bubble sort algorithm. It does not change the node links, only the data inside the nodes.

Question: Write a program to sort the elements of a singly linked list in ascending order.

```

#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

void sortAscending(struct Node* head) {
    struct Node *i, *j;
    int temp;
    for (i = head; i != NULL; i = i->next) {
        for (j = i->next; j != NULL; j = j->next) {
            if (i->data > j->data) {
                temp = i->data;
                i->data = j->data;
                j->data = temp;
            }
        }
    }
}

```

```

void displayList(struct Node* head) {
    while (head != NULL) {
        printf("%d -> ", head->data);
        head = head->next;
    }
    printf("NULL\n");
}

struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}

int main() {
    struct Node* head = createNode(30);
    head->next = createNode(10);
    head->next->next = createNode(20);
    printf("Before sorting: ");
    displayList(head);
    sortAscending(head);
    printf("After sorting: ");
    displayList(head);
    return 0;
}

```

Function 3: Merge Two Sorted Linked Lists

This program merges two already sorted linked lists into a single sorted linked list using recursion. It compares node values and links them in order without creating new nodes.

Question: Write a program to merge two already sorted linked lists into one sorted linked list.

```

#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;

```

```

    newNode->next = NULL;
    return newNode;
}

void displayList(struct Node* head) {
    while (head != NULL) {
        printf("%d -> ", head->data);
        head = head->next;
    }
    printf("NULL");
}

struct Node* mergeLists(struct Node* l1, struct Node* l2) {
    if (!l1) return l2;
    if (!l2) return l1;

    if (l1->data < l2->data) {
        l1->next = mergeLists(l1->next, l2);
        return l1;
    } else {
        l2->next = mergeLists(l1, l2->next);
        return l2;
    }
}

int main() {
    struct Node* list1 = createNode(10);
    list1->next = createNode(20);
    list1->next->next = createNode(30);

    struct Node* list2 = createNode(15);
    list2->next = createNode(25);
    list2->next->next = createNode(35);

    printf("List 1: ");
    displayList(list1);
    printf("List 2: ");
    displayList(list2);

    struct Node* merged = mergeLists(list1, list2);
    printf("Merged List: ");
    displayList(merged);

    return 0;
}

```

Function 4: Shortlist Even Numbers

This function traverses the linked list and prints only the even-valued nodes. It is useful to show how conditional filtering can be applied when accessing list data.

Question: Write a program to print only the even numbers from a singly linked list.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}

void shortlistEven(struct Node* head) {
    printf("Filtered List (Even Numbers): ");
    while (head != NULL) {
        if (head->data % 2 == 0) {
            printf("%d -> ", head->data);
        }
        head = head->next;
    }
    printf("NULL");
}

int main() {
    struct Node* head = createNode(5);
    head->next = createNode(8);
    head->next->next = createNode(13);
    head->next->next->next = createNode(22);

    shortlistEven(head);

    return 0;
}
```

Function 5: Count Occurrences of a Given Value

This program counts how many times a specific value occurs in the linked list. It demonstrates the concept of traversing a list and using a counter to track matching values.

Question: Write a program to count the number of occurrences of a given value in a singly linked list.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}

int countOccurrences(struct Node* head, int value) {
    int count = 0;
    while (head != NULL) {
        if (head->data == value) {
            count++;
        }
        head = head->next;
    }
    return count;
}

int main() {
    struct Node* head = createNode(10);
    head->next = createNode(20);
    head->next->next = createNode(10);
    head->next->next->next = createNode(30);

    int value = 10;
    int result = countOccurrences(head, value);
    printf("Value %d occurs %d times in the list.", value, result);

    return 0;
}
```

Function 6: Reverse a Linked List

This function reverses the order of nodes in a singly linked list by adjusting the links between them. It helps students understand pointer manipulation and memory direction.

Question: Write a program to reverse a singly linked list.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}

struct Node* reverseList(struct Node* head) {
    struct Node* prev = NULL;
    struct Node* curr = head;
    struct Node* next = NULL;
    while (curr != NULL) {
        next = curr->next;
        curr->next = prev;
        prev = curr;
        curr = next;
    }
    return prev;
}

void displayList(struct Node* head) {
    while (head != NULL) {
        printf("%d -> ", head->data);
        head = head->next;
    }
    printf("NULL\n");
}

int main() {
    struct Node* head = createNode(10);
    head->next = createNode(20);
}
```

```

head->next->next = createNode(30);

printf("Original List: ");
displayList(head);

head = reverseList(head);
printf("Reversed List: ");
displayList(head);

return 0;
}

```

Function 7: Remove Duplicates from Sorted List

This function removes duplicate nodes from a sorted linked list. It checks consecutive nodes and removes the second if both have the same value.

Question: Write a program to remove duplicate elements from a sorted linked list.

```

#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}

void removeDuplicates(struct Node* head) {
    struct Node* current = head;
    while (current != NULL && current->next != NULL) {
        if (current->data == current->next->data) {
            struct Node* temp = current->next;
            current->next = temp->next;
            free(temp);
        } else {
            current = current->next;
        }
    }
}

```



```

void displayList(struct Node* head) {
    while (head != NULL) {
        printf("%d -> ", head->data);
        head = head->next;
    }
    printf("NULL
");
}

int main() {
    struct Node* head = createNode(10);
    head->next = createNode(10);
    head->next->next = createNode(20);
    head->next->next->next = createNode(30);
    head->next->next->next->next = createNode(30);

    printf("Original List with Duplicates: ");
    displayList(head);

    removeDuplicates(head);
    printf("List after Removing Duplicates: ");
    displayList(head);

    return 0;
}

```

Function 8: Free the Linked List

This program deallocates memory for each node in the linked list to avoid memory leaks. It demonstrates the importance of cleanup in programs that use dynamic memory allocation.

Question: Write a program to deallocate memory used by a singly linked list using a function.

```

#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = NULL;
}

```

```

        return newNode;
    }

void freeList(struct Node* head) {
    struct Node* temp;
    while (head != NULL) {
        temp = head;
        head = head->next;
        free(temp);
    }
}

int main() {
    struct Node* head = createNode(10);
    head->next = createNode(20);
    head->next->next = createNode(30);

    printf("Freeing the list...");
    freeList(head);
    printf("Memory freed successfully.");

    return 0;
}

```

Summary Points for Students

- Sorting can be done by data swapping, not just pointer rearranging.
- Merging maintains order when lists are already sorted.
- Filtering (shortlisting) shows how to extract information.
- Removing duplicates helps with data sanitization.
- Counting is useful for frequency analysis.
- Always free allocated memory to prevent memory leaks.

- End -